Basics Day 6 — Session 6 (Hours 21–24)

Python Programming (Basic) • Choosing Structures, Drills, Tracker & Checkpoint

Session 6 Overview

Topics Covered Today

- Hour 21: Choosing the right structure — list, tuple, set, dict
- Hour 22: Data-structure drill circuit (guided practice)
- Hour 23: Mini-project — in-memory book tracker
- Hour 24: Checkpoint 3 — data structures assessment

Hour 21: Choosing the Right Structure

Learning Outcomes

- Explain in plain language what problem each core structure solves best
- Choose list vs set based on whether order and duplicates matter
- Choose list vs dictionary based on whether key lookup matters
- Explain when a tuple fits better than a list for a small, fixed record
- Refactor a working solution to a better-fitting structure and explain one tradeoff

The Four Core Structures

Structure	Ordered?	Mutable?	Duplicates?	Best For
List	Yes	Yes	Allowed	sequences that change
Tuple	Yes	No	Allowed	small fixed records
Set	No	Yes	Not allowed	uniqueness and membership
Dictionary	Insertion order	Yes	Keys must be unique	lookup by key

> Source: Day6_Hour1_Basics.md §4.5 — one-minute comparison table

Decision Checklist

Five Questions Before You Choose

1. **Does order matter?** — Yes → List or Tuple
2. **Are duplicates allowed?** — No → Set
3. **Do I need lookup by a meaningful key?** — Yes → Dictionary
4. **Is this one small fixed record?** — Yes → Tuple
5. **Will this collection change over time?** — No → Tuple; Yes → List / Set / Dict

> Source: Day6_Hour1_Basics.md §5.6 — decision checklist

Demo: Duplicate Detection — List First, Then Set

```
usernames = ["ava", "liam", "ava", "mia", "liam", "noah"]
seen = []

for username in usernames:
    if username in seen:
        print(f"Duplicate found: {username}")
    else:
        seen.append(username)
        print(f"New user: {username}")

print(f"Seen users: {seen}")
```

Why consider a refactor?

- Are we using the order in `seen` for anything? No.
- Do we want duplicates in `seen`? No.
- Are we mainly using `in` for membership? Yes.

Refactored: Set Expresses the Intent

```
usernames = ["ava", "liam", "ava", "mia", "liam", "noah"]
seen: set[str] = set()

for username in usernames:
    if username in seen:
        print(f"Duplicate found: {username}")
    else:
        seen.add(username)
        print(f"New user: {username}")

print(f"Seen users: {seen}")
```

Same behavior — better structure choice. The set now expresses the rule of uniqueness directly.

> Source: Day6_Hour1_Basics.md §7.4

Demo: Phone Lookup — List First, Then Dictionary

```python
contacts: list[tuple[str, str]] = [
    ("Ava", "555-0111"),
    ("Liam", "555-0222"),
    ("Mia", "555-0333"),
]

search_name: str = input("Enter a contact name: ")
found_phone = None

for name, phone in contacts:
    if name == search_name:
        found_phone = phone
        break

if found_phone is None:
    print("Contact not found")
else:
    print(f"{search_name}: {found_phone}")
```

Refactored: Dictionary Makes Lookup Direct

```
contacts: dict[str, str] = {
    "Ava": "555-0111",
    "Liam": "555-0222",
    "Mia": "555-0333",
}

search_name: str = input("Enter a contact name: ")
phone: str = contacts.get(search_name, "Contact not found")
print(f"Result: {phone}")
```

One line, clear meaning, safe missing-key handling with `get()`.

> Source: Day6_Hour1_Basics.md §8.3–§8.4

Lab: Refactor Challenge

Time: 13 minutes

Part A — Duplicate Checker

```
raw_names = ["Ana", "Ben", "Ana", "Chris", "Ben", "Dana"]
```

1. Build a list-based version using `seen_names` to report duplicates
2. Refactor to use a set instead
3. Print the final unique names
4. Add a comment explaining why the set version is a better fit

Part B — Score Lookup

```
score_pairs = [("Ava", 91), ("Liam", 88), ("Mia", 95)]
```

Common Pitfalls — Hour 21

⚠️ Empty set written as `{}` accidentally creates a dictionary — use `set()` instead
⚠️ Using `append()` on a set — sets use `add()`
⚠️ Refactoring the structure but forgetting to change the method calls
⚠️ Assuming the set will print in the same order as the original list
⚠️ Explaining only that one version is "better" without saying *why*

> Source: Day6_Hour1_Basics.md §9.5

Quick Check — Hour 21

Question: If I need to check whether a name has already been seen, and order doesn't matter, which structure is the better fit — a list or a set?

Hour 22: Data-Structure Drill Circuit

Learning Outcomes

- Complete short practice tasks using lists, tuples, sets, and dicts with increasing confidence
- Recognize the core pattern behind each structure quickly
- Test small pieces of code with sample values before deciding a task is finished
- Explain verbally why a given structure fits a specific short problem
- Share one working solution and describe one mistake you corrected

Circuit Format

Four Stations — Work Through Them in Order

1. **Station 1:** Filter numbers > 10 — list pattern
2. **Station 2:** Unique emails — set pattern
3. **Station 3:** Word frequency — dictionary pattern
4. **Station 4:** Coordinates unpacking — tuple pattern

Success Target

Complete at least **three of four** stations. At each station: code runs, output matches the prompt, and you can explain why the structure fits.

> Source: Day6_Hour2_Basics.md §4.1–§4.2

Station 1: Filter Numbers > 10 (List)

```
numbers = [4, 11, 9, 15, 22, 3, 10, 18]
filtered: list[int] = []

for number in numbers:
    if number > 10:
        filtered.append(number)

print(filtered)
```

Tasks

1. Create a new list containing only numbers greater than 10
2. Print the filtered list
3. Print how many numbers made it into the filtered list (`len()`)

Completion Criteria

Station 2: Unique Emails (Set)

```
emails = [
    "ana@example.com",
    "ben@example.com",
    "ana@example.com",
    "chris@example.com",
    "ben@example.com",
    "dana@example.com",
]
```

Tasks

1. Convert the list into a set of unique email addresses
2. Print the unique emails
3. Print the original count and the unique count
4. Add a short comment explaining why a set fits this task

Station 3: Word Frequency (Dictionary)

```
words = ["red", "blue", "red", "green", "blue", "red"]
counts: dict[str, int] = {}

for word in words:
    counts[word] = counts.get(word, 0) + 1

print(counts)
for word, count in counts.items():
    print(f"{word}: {count}")
```

Tasks

1. Build a dictionary counting how many times each word appears
2. Print the dictionary
3. Print each word and count on its own line
4. Add a comment explaining why a dictionary fits better than a list

Station 4: Coordinates Unpacking (Tuple)

```
points = [(2, 3), (5, 8), (1, 4), (7, 2)]
largest_x = None

for x, y in points:
    print(f"Point at x={x}, y={y}")
    if largest_x is None or x > largest_x:
        largest_x = x

print(f"Largest x value: {largest_x}")
```

Tasks

1. Loop through the points
2. Unpack each tuple into `x` and `y`
3. Print each point in a sentence
4. Track and print the largest `x` value

Lab: Drill Circuit

Time: ~36 minutes (8–9 minutes per station)

Your Goal

Work through all four stations in order. Use this rhythm at every station:

1. Read the prompt — choose the right structure
2. Build the smallest working version you can
3. Test with tiny data
4. Improve output if time remains

If You Finish Early

Print numbers one per line with a label; add a sorted station-2 result with `sorted()`; ask the user for a color in station 3 and print its count using `get()`

Common Pitfalls — Hour 22

⚠️ Station 1 — appending to the original list instead of the result list
⚠️ Station 2 — creating `{}` and making an empty dictionary instead of a set
⚠️ Station 2 — expecting the set to preserve the original order
⚠️ Station 3 — using `counts[word] += 1` before the key exists (KeyError)
⚠️ Station 3 — forgetting `.items()` when printing both word and count
⚠️ Station 4 — writing `for point in points:` and forgetting to unpack

> Source: Day6_Hour2_Basics.md §6.6, §7.6, §8.6, §9.6

Quick Check — Hour 22

Question: What does this line do, and why is it better than `counts[word] += 1`?

```
counts[word] = counts.get(word, 0) + 1
```

Hour 23: Mini-Project — In-Memory Tracker

Learning Outcomes

- Explain why a list of dictionaries fits a small in-memory tracker with multiple records
- Build a tracker that stores records, adds a new record, lists all records, and searches
- Read and write record fields using clear dictionary keys
- Loop through a list of records and format output cleanly with f-strings
- Describe one reason for choosing this storage model

Choosing the Storage Model

Option 1: Plain list of strings

```
books = ["Dune", "1984", "The Hobbit"]
```

Simple, but stores only one piece of information per book.

Option 2: Dictionary from title to author

```
books = {"Dune": "Frank Herbert", "1984": "George Orwell"}
```

Better for lookup by title, but where does genre go?

Option 3: List of dictionaries ✅

```
books = [
    {"title": "Dune", "author": "Frank Herbert", "genre": "Science Fiction"},
    {"title": "1984", "author": "George Orwell", "genre": "Dystopian"},
```

Starter Data and Program Shape

```
books: list[dict[str, str]] = [
    {"title": "Dune", "author": "Frank Herbert", "genre": "Science Fiction"},
    {"title": "1984", "author": "George Orwell", "genre": "Dystopian"},
    {"title": "The Hobbit", "author": "J.R.R. Tolkien", "genre": "Fantasy"},
]
```

Three Required Features

1. **Add** a book
2. **List** all books
3. **Search** for a book by title

Design rule: outer structure = list (holds many records); inner structure = dictionary (holds one record's named fields)

> Source: Day6_Hour3_Basics.md §5.1–§5.2

Add Feature

```
new_title: str = input("Enter a book title: ")
new_author: str = input("Enter the author: ")
new_genre: str = input("Enter the genre: ")

new_book: dict[str, str] = {
    "title": new_title,
    "author": new_author,
    "genre": new_genre,
}

books.append(new_book)
print("Book added.")
```

Structure: ask for values → build one dictionary → append to the outer list

> Source: Day6_Hour3_Basics.md §6.2–§6.3

List Feature — User-Friendly Output

```
print("\nLibrary books:")
for book in books:
    print(f"- {book['title']} by {book['author']} ({book['genre']})")
```

Debug output vs. user-friendly output

- **Debug:** `print(books)` — prints raw list of dicts; useful for inspecting
- **User-friendly:** loop with f-string — readable by a person

Habit: move from debug printing to formatted output as the final step.

> Source: Day6_Hour3_Basics.md §7.2–§7.3

Search Feature

```
search_title: str = input("\nEnter a title to search for: ")
found: bool = False

for book in books:
    if book["title"].lower() == search_title.lower():
        print("Found book:")
        print(f"  Title:  {book['title']}")
        print(f"  Author: {book['author']}")
        print(f"  Genre:  {book['genre']}")
        found = True
        break

if not found:
    print("Book not found.")
```

Key details: `.lower()` on both sides (case-insensitive); `found` flag for the not-found message; `break` once found.

Lab: Book Library Tracker

Time: 19 minutes

Task

Build a small Python program that stores books using a **list of dictionaries**.

Each book record must include `title`, `author`, and `genre`. Your program must:

1. Start with at least **three** books already in the list
2. Ask the user for one new book and add it
3. Print all books in a clean, readable format
4. Ask the user for a title to search — print the matching book or "Book not found."

Completion Criteria

✓ Uses at least one list and one dictionary meaningfully

Common Pitfalls — Hour 23

- ⚠️ Storing each new book as a list instead of a dictionary
- ⚠️ Using different key names in different records (`name` in one, `title` in another)
- ⚠️ Printing the raw list and thinking the formatting requirement is complete
- ⚠️ Forgetting to set or check the `found` flag in the search logic
- ⚠️ Comparing titles without normalizing case and assuming search is broken
- ⚠️ Creating the new book dictionary but forgetting to append it to `books`

> Source: Day6_Hour3_Basics.md §9.6

Quick Check — Hour 23

Question: Why is a list of dictionaries a good fit for this tracker?

Expected answer: "I used a list of dictionaries because I had many books, and each book had multiple named fields. The list lets me store many records, and the dictionary makes each record readable."

Hour 24: Checkpoint 3 — Data Structures Assessment

Learning Outcomes

- Demonstrate confident use of a dictionary for real-world data storage
- Apply safe dictionary access patterns (`get()` or `in` checks) to avoid KeyError crashes
- Format and display structured data clearly and readably
- Test programs systematically — check both successful searches and failure cases
- Reflect on data-structure selection and explain why a dictionary fits this task

What This Checkpoint Tests

Five Skills

1. **Can you choose the right data structure?** — A dictionary is the answer: given a name, find a phone
2. **Can you create and populate a structure cleanly?** — Five contacts, five lines
3. **Can you print data in a form a human can read?** — Not just `print(contacts)`
4. **Can you accept and handle user input?** — The `input()` function
5. **Can you handle the case where something is missing?** — No KeyError; respond gracefully

> Source: Day6_Hour4_Basics.md §3.2

Safe Key Access — Two Patterns

Pattern 1: Check First with `in`

```
search_name = input("Enter name: ")
if search_name in contacts:
    print(contacts[search_name])
else:
    print("Contact not found.")
```

Pattern 2: Use `get()` with a Default

```
search_name = input("Enter name: ")
phone = contacts.get(search_name, "Contact not found.")
print(phone)
```

Both patterns are safe. If a searched name is missing, your program must not crash.

> Source: Day6_Hour4_Basics.md §4.2

Checkpoint Build Order

Step-by-Step Approach

```
# Step 1: Create the dictionary with five starting contacts
contacts = {
    'Alice': '555-1234',
    'Bob': '555-5678',
    'Carol': '555-9999',
    'Diana': '555-4444',
    'Eve': '555-5555'
}

# Step 2: Display all contacts (loop with .items())

# Step 3: Ask the user for a name to search

# Step 4: Search safely (in check or .get())

# Step 5: Test — one name that exists, one that does not
```

Checkpoint Task: Simple Contacts (In Memory)

Name: Simple Contacts (In Memory)

Minimum Features

- Create a dictionary with at least **5 contacts** (name → phone)
- **Display all contacts** in readable format — one per line with labels
- **Accept user input** to search for a contact by name
- Return the phone number if contact exists; print a **friendly message** if not
- **Critical:** Handle missing keys safely — no KeyError crash

Readable Output Example

```
— All Contacts —
Alice: 555-1234
Bob: 555-5678
Carol: 555-9999
```

Test Plan for the Checkpoint

```
Test 1: Display all contacts
  Expected: All 5 contacts printed clearly, one per line

Test 2: Search for a contact that exists
  Example: Search for "Alice"
  Expected: Print Alice's phone number

Test 3: Search for a contact that does NOT exist
  Example: Search for "Zzzzzz"
  Expected: Print "Contact not found" — no crash

Test 4 (optional): Case sensitivity
  Example: Search for "alice" when stored as "Alice"
  Expected: Either finds it or reports not found
```

> Source: Day6_Hour4_Basics.md §5.2

Lab: Checkpoint 3

Time: 25–35 minutes of uninterrupted work

Task

Build the Simple Contacts program following the build order above.

Completion Criteria

✓ Dictionary created with at least 5 entries
✓ Loop printing contacts one per line (not raw dictionary dump)
✓ Safe search code — either `in` check or `.get()`
✓ At least one test run with a name that exists
✓ At least one test run with a name that does not exist
✓ Friendly message for missing contacts — no crash

> Source: Day6_Hour4_Basics.md §7.1–§7.3

Common Pitfalls — Hour 24

⚠️ Bare bracket access `contacts[search_name]` without a safety check — causes KeyError

⚠️ Printing `print(contacts)` as final output instead of a formatted loop

⚠️ Swapping key and value in `.items()` loop — `for phone, name in contacts.items():`

⚠️ Testing only the success case and never checking a missing name

⚠️ Case sensitivity confusion — "Alice" and "alice" are different keys

⚠️ Adding extra features before the minimum requirements work

> Source: Day6_Hour4_Basics.md §8.1–§8.6

Quick Check — Hour 24

Question: What is the difference between `contacts[search_name]` and `contacts.get(search_name, "Not found")`?

Expected answer: "`contacts[search_name]` raises a KeyError if the key is missing. `contacts.get(search_name, 'Not found')` returns the default value instead of crashing, making it safe for user-provided input."

Session 6 Wrap-Up

What We Covered Today

Hour 21 — Choosing the Right Structure

- Decision checklist: order, duplicates, key lookup, fixed records, mutability
- Refactor challenge: list → set, list-of-tuples → dictionary

Hour 22 — Drill Circuit

- Four patterns: list filtering, set uniqueness, dictionary counting, tuple unpacking
- Testing rhythm: start small, test first, explain your choice

Hour 23 — Mini-Project

- List of dictionaries as storage model

Scope Guardrail Reminder

Stay in Basics Scope

✓ Core structures: list, tuple, set, dictionary
✓ Safe key access: `in` check, `get()` with default
✓ Formatted output using f-strings and loops
✓ Simple in-memory storage (no file I/O yet)

Not Yet (Advanced Topics)

✗ List comprehensions
✗ Lambda functions
✗ Big O notation or performance analysis
✗ Custom classes or data modeling
✗ Nested dictionaries beyond list-of-dicts
✗ External packages or databases

Next Session Preview

Session 7 (Hours 25–28)

- Hour 25: Conditionals in depth — `if/elif/else`, boundaries, compound conditions
- Hour 26: Defining and calling functions
- Hour 27: Parameters, return values, and scope
- Hour 28: Checkpoint 4 — functions + data structures

Coming Skills

Writing reusable functions and structuring programs with clean logic flow

Questions?

Remember:

- Choose structures based on the job, not your favorite
- Always test both the success path and the failure path
- User-friendly output is not optional — it is a programming habit
- Safe key access prevents crashes in real programs

Thank You!

See you in Session 7!